

# 1bit-CPU設計 (TR10)

---

ISHI会

<https://ishi-kai.org/>

Mail: [info@ishi-kai.org](mailto:info@ishi-kai.org)

# もくじ： デジタル回路編

- 論理回路（ゲート回路）
- 加算回路（演算器）

# もくじ： レイアウト生成編

- 論理合成
- 生成フロー
- 論理合成・詳解
- P & R（配置配線）
- タイミング解析

# もくじ： 1bit-CPU編

- CPUの構成
- 回路図
- レイアウト

# デジタル回路編

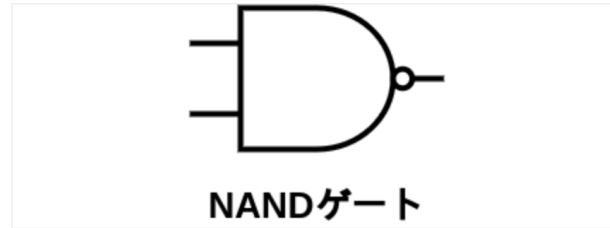
出典:<https://vlsi.jp/>



# 論理回路（ゲート回路）

## NAND

次は**NANDゲート**、これはANDゲートの出力にNOTしたものです、入力の少なくともどちらか一方が0なら1を出力します。それ以外なら1を出力します。



真理値表は以下の通りです。

| A | B | A NAND B |
|---|---|----------|
| 0 | 0 | 1        |
| 0 | 1 | 1        |
| 1 | 0 | 1        |
| 1 | 1 | 0        |

NANDゲートの真理値表

Verilogでは、NANDはNOTゲートとANDゲートを組み合わせて **$\sim(\text{信号名1} \ \& \ \text{信号名2})$** で表せます。カッコ()は数式と同じく優先順位を表しており、カッコの中身が優先され先に実行されます。少し難易度が上がりましたかね？実際に使ってみましょう。

**Basic.v**のassign文の部分を編集し、ANDゲートをNANDゲートに変更します。

```
assign w_x = ~(r_a & r_b);
```

### Libraries

LIB - IP62\_5\_stdcell.gds

AND2\_X1  
AND3\_X1  
AND4\_X1  
BUF\_X1  
BUF\_X12  
BUF\_X16  
BUF\_X2  
BUF\_X4  
BUF\_X8  
CLKBUF\_X1  
CLKBUF\_X12  
CLKBUF\_X16  
CLKBUF\_X2  
CLKBUF\_X4  
CLKBUF\_X8  
DEL1  
DEL2  
DEL4  
DFFR  
DFFS  
INV\_X1  
INV\_X12  
INV\_X16  
INV\_X2  
INV\_X4  
INV\_X8  
MUX2  
NAND2  
NAND3  
NAND4  
NOR2  
NOR3  
NOR4  
OR2  
OR3  
OR4  
XNOR2  
XOR2

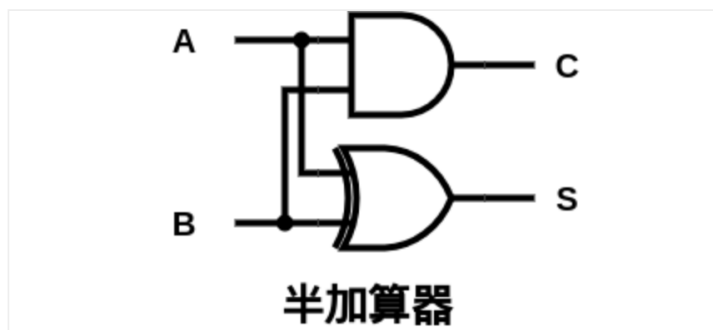


# 加算回路（演算器）



## HalfAdder

次はデジタル回路で少し面白い回路を紹介します。HalfAdder、**半加算器**です。以下のANDとXORで構成された回路を見てください。



この回路の真理値表を書いてみましょう。以下の通りです。

| A | B | C | S |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

CがAND回路の出力でSがXOR回路の出力になっています。当然っちゃ当然ですね。ですがよく見てください、このCとS、AとBの二進数の加算結果の二桁目と一桁目になっていますね。ちなみにCは繰り上がりを意味するCarryの略で、Sは和を意味するSumの略です、

上記の表と下の数式を見比べればよく分かります。

$$0_{(2)} + 0_{(2)} = 00_{(2)}$$

$$0_{(2)} + 1_{(2)} = 01_{(2)}$$

$$1_{(2)} + 0_{(2)} = 01_{(2)}$$

$$1_{(2)} + 1_{(2)} = 10_{(2)}$$

この様に加算と同じ動作をする回路のことを**加算器**と呼びます。覚えておきましょう。

それではVerilogで半加算器を作成してみます。まずはBasic.vにORゲートの検証の時に使ったコードをコピーしてください。

そして信号線w\_xの宣言を消し、新たにCarryとSumを入力する信号線w\_cとw\_sを定義します。

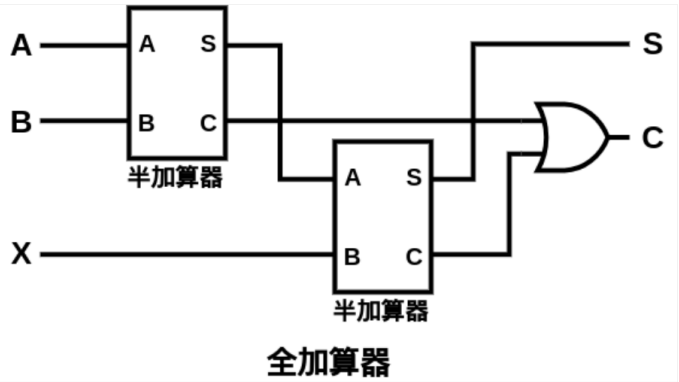
```
wire w_c;  
wire w_s;
```

続いてassign文で図と同じようにANDとXORを用いて半加算器を作成します。

```
assign w_c = r_a & r_b;  
assign w_s = r_a ^ r_b;
```

# FullAdder

さて、半加算器は二進数の1桁同士の加算を行う回路でした。更に2桁、10桁の加算を行いたい時、この半加算器を並べるだけでは複数桁の加算を行う事は出来ません。加算をやった事のある皆さんはご存知でしょうが、加算は繰り上がりが発生する演算です。よって、この半加算器を繰り上がり入力を受け取るように拡張する必要があります。その回路を全加算器、FullAdderと呼びます。実際に見てみましょう。



半加算器2つとORで構成されていますね。新しく追加された全加算器の入力のXは繰り上がり入力を意味します。真理値表を見てみましょう。

| A | B | X | C | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

入力のA, B, Xと出力のC, Sを下の二進数の加算の式を見比べてみましょう。一致していますね。

$$Q_{(2)} + 0_{(2)} + 0_{(2)} = 0Q_{(2)}$$

$$Q_{(2)} + 0_{(2)} + 1_{(2)} = 01_{(2)}$$

$$Q_{(2)} + 1_{(2)} + 0_{(2)} = 01_{(2)}$$

$$Q_{(2)} + 1_{(2)} + 1_{(2)} = 1Q_{(2)}$$

$$1_{(2)} + 0_{(2)} + 0_{(2)} = 01_{(2)}$$

$$1_{(2)} + 0_{(2)} + 1_{(2)} = 1Q_{(2)}$$

$$1_{(2)} + 1_{(2)} + 0_{(2)} = 1Q_{(2)}$$

$$1_{(2)} + 1_{(2)} + 1_{(2)} = 11_{(2)}$$

それでは実際に全加算器をVerilogで作成していきます。回路を作る前にまずはBasic.vに手を加えます。以下のコードをBasic.vにコピーしてください。

```
module Basic;

// --- おまじないここから ---
initial begin
    $dumpfile("wave.vcd");
    $dumpvars(0, Basic);
end

reg r_a;
reg r_b;
reg r_x;
// --- おまじないここまで ---

wire w_c;
wire w_s;

// ここに全加算器を記述
```

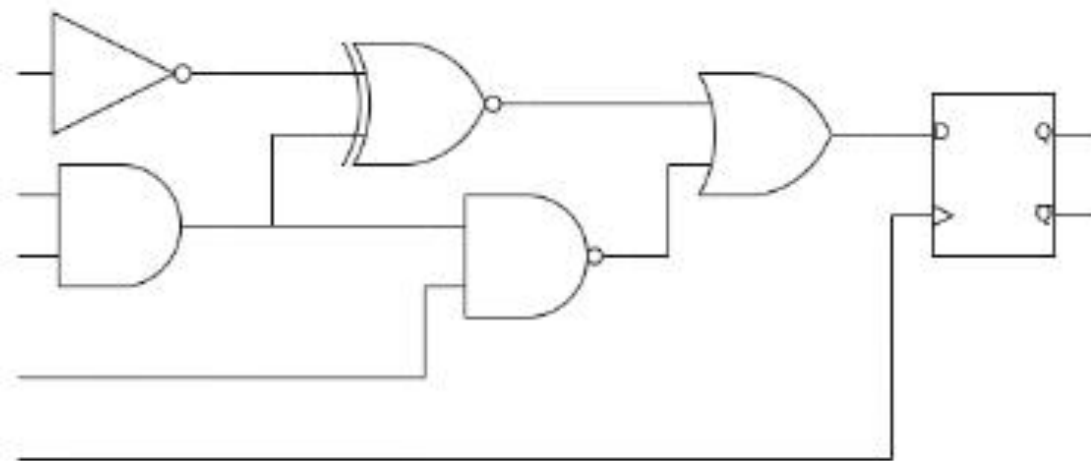
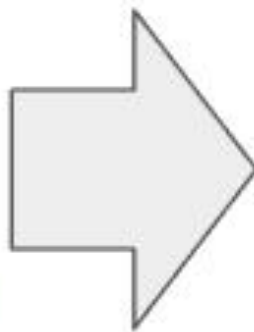
# レイアウト生成編



# 論理合成



**HDL**



**デジタル回路**

**論理合成のイメージ**

論理合成

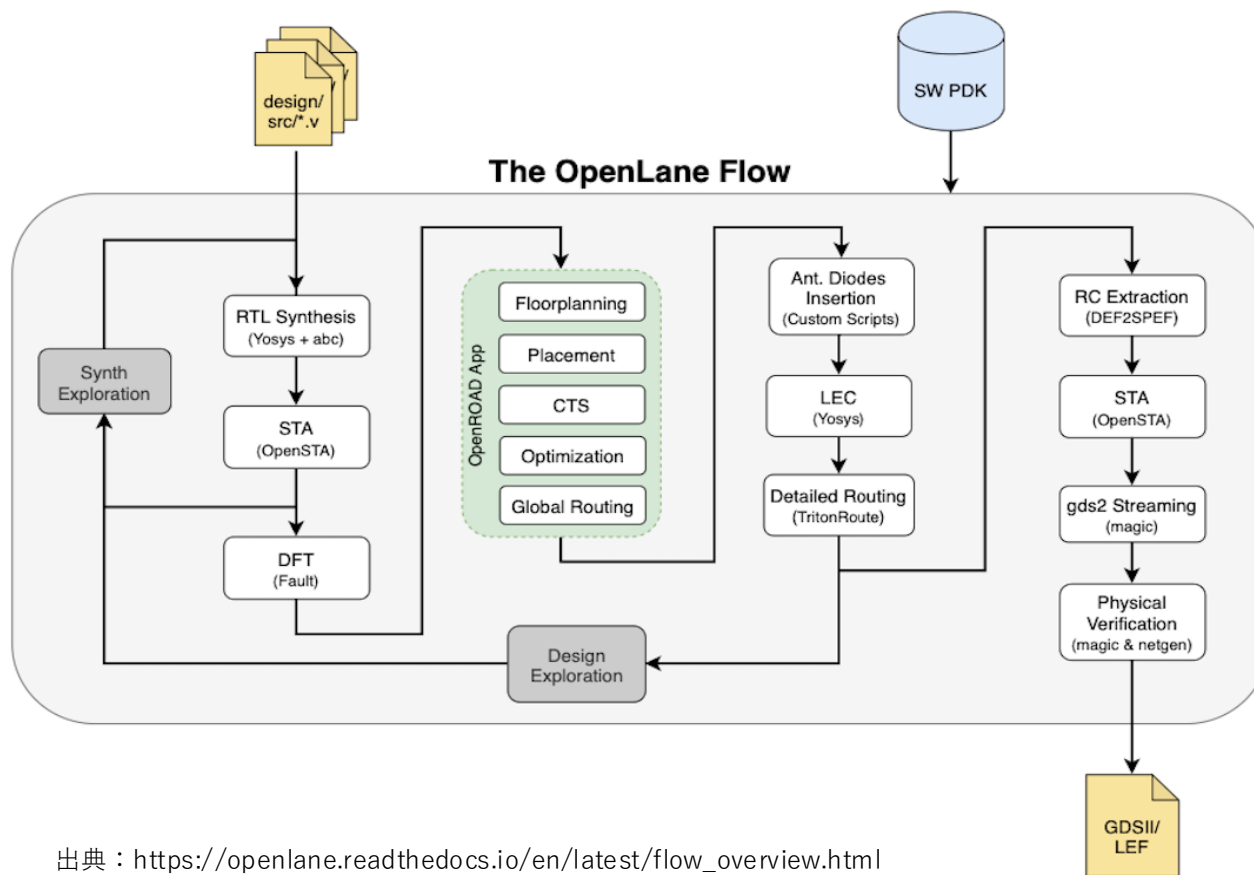
HDL（ハードウェア記述言語）で記述して、論理回路（RTL）にする



# 生成フロー



# OpenLANEによる生成フロー



出典 : [https://openlane.readthedocs.io/en/latest/flow\\_overview.html](https://openlane.readthedocs.io/en/latest/flow_overview.html)

# 生成フロー詳細

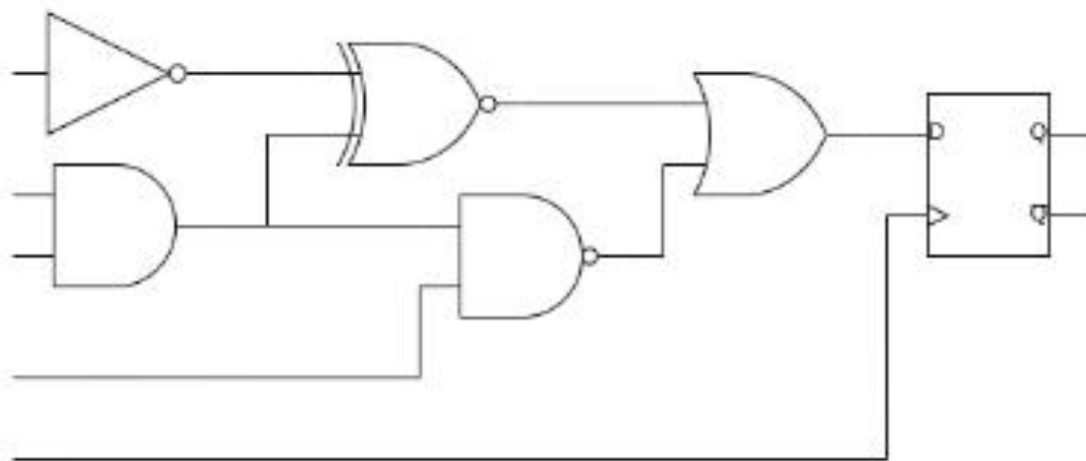
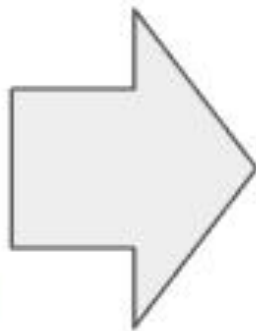
| 合成      |           | フロアプランと電源供給 |                | 配置      |         | クロックツリー合成と配線    |             | GDS生成とチェック |                                   |
|---------|-----------|-------------|----------------|---------|---------|-----------------|-------------|------------|-----------------------------------|
| ツール名    | 機能内容      | ツール名        | 機能内容           | ツール名    | 機能内容    | ツール名            | 機能内容        | ツール名       | 機能内容                              |
| yosys   | RTLを論理合成  | init_fp     | コア領域の定義        | RePLace | グローバル配置 | TritonCTS       | クロックツリーの合成  | Magic      | GDSIIファイル生成                       |
| abc     | PDKマッピング  | ioplacer    | 入出力ポートの設置      | Resizer | 最適化     | FastRouteとCU-GR | グローバル配線     | Magic      | DRC (Design Rule Check) とアンテナチェック |
| OpenSTA | 静的タイミング解析 | pdn         | 給電ネットワークの生成    | OpenDP  | ローカル配置  | TritonRoute     | ローカル配線      | Netgen     | LVS (Layout vs Schematic) チェック    |
|         |           | tapcell     | タップとデキャップセルの挿入 |         |         | SPEF_Extractor  | 寄生フォーマットの抽出 | CVC        | 回路妥当性検証                           |



# 論理合成・詳解



**HDL**



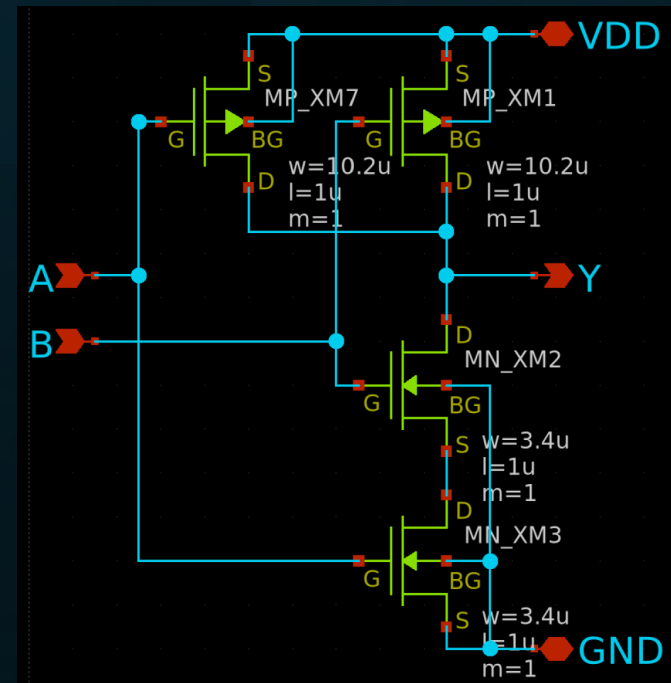
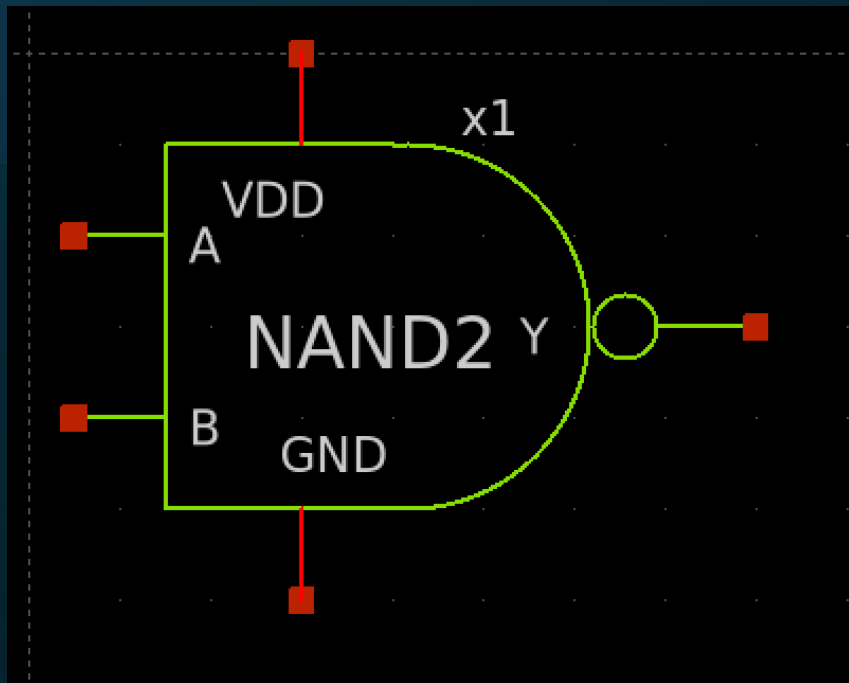
**デジタル回路**

**論理合成のイメージ**

論理合成

HDL（ハードウェア記述言語）で記述して、論理回路（RTL）にする

# 論理回路の正体

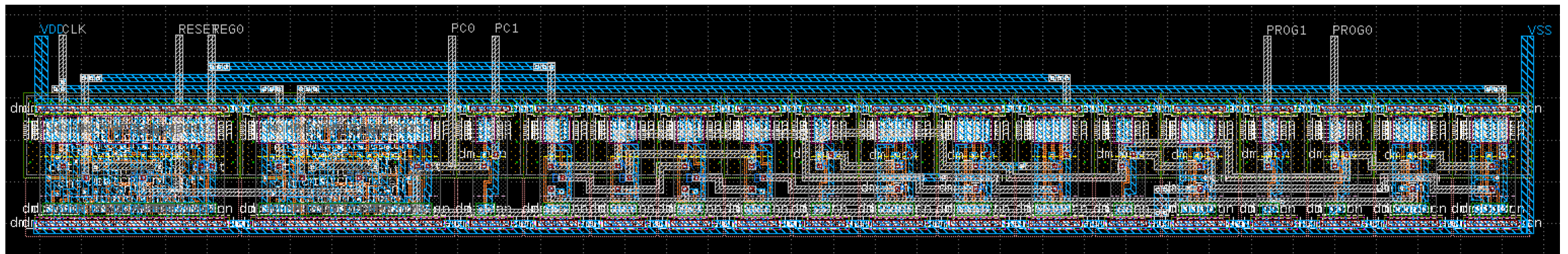
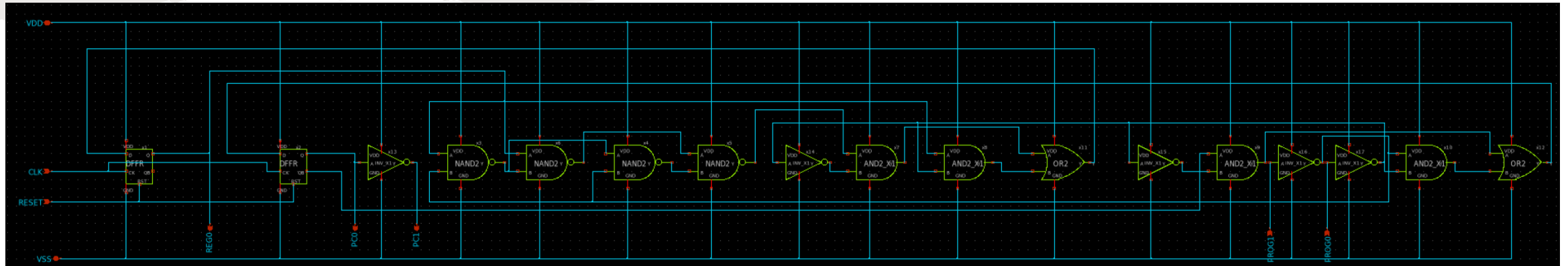


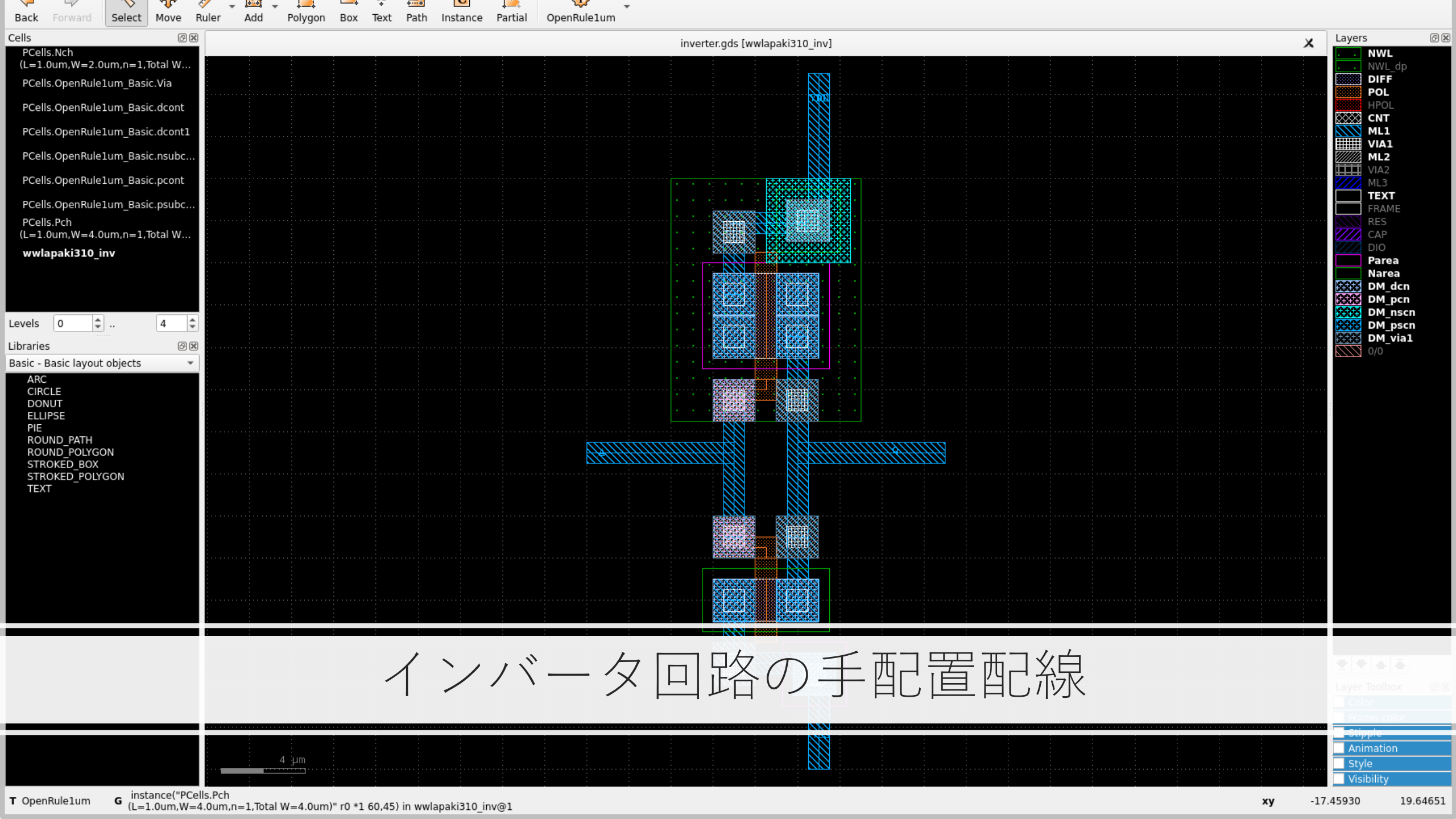
中身はP-FETとN-FET



P&R（配置配線）

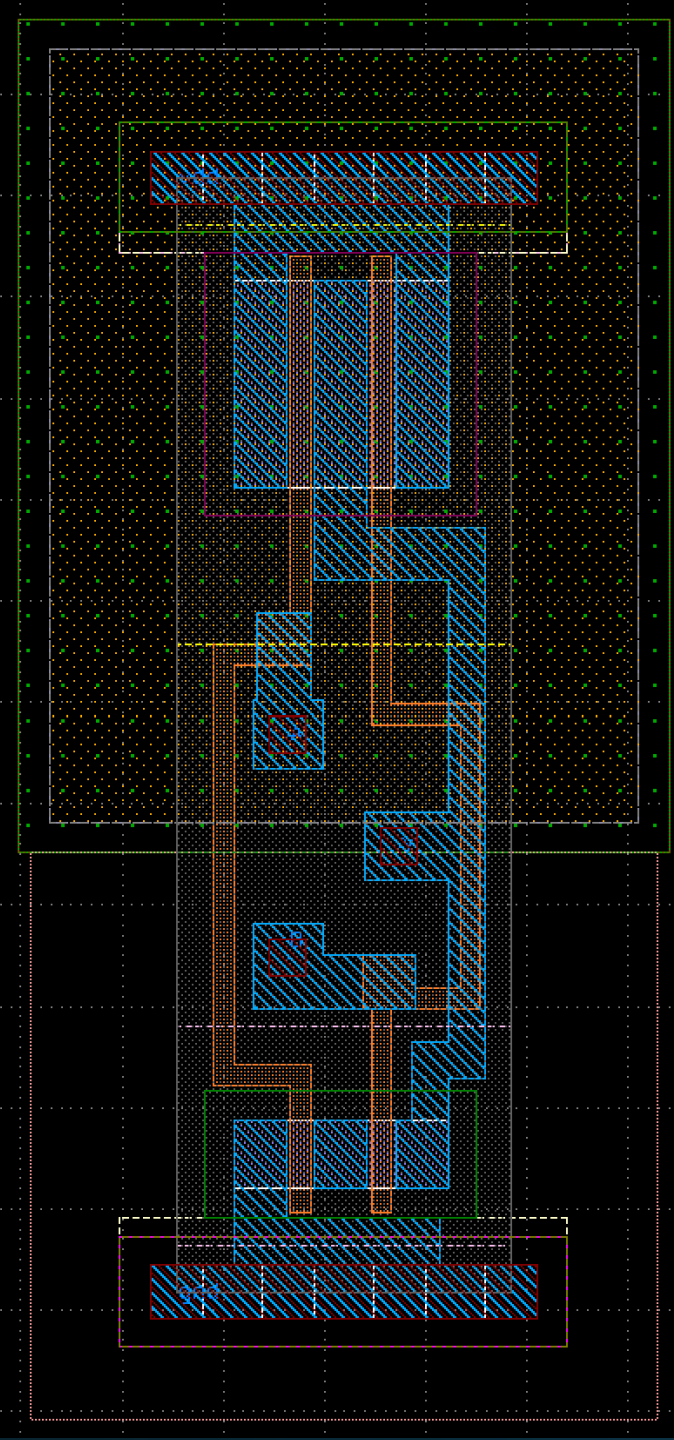
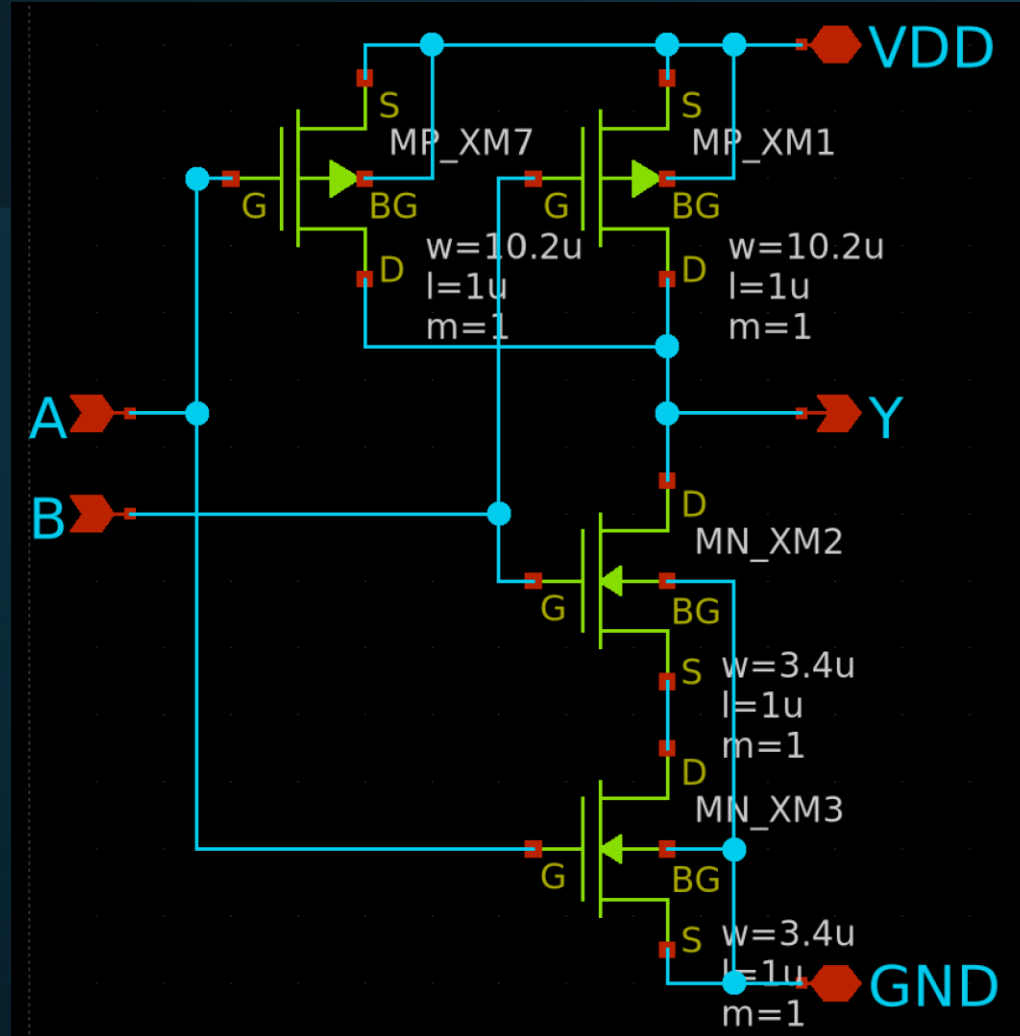
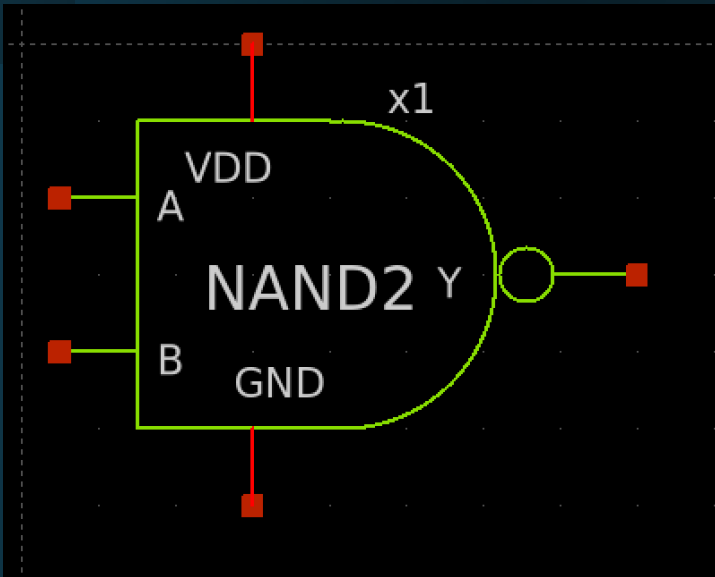
# 回路をレイアウトにする工程







# スタンダードセル



- 縦の長さは均一
- 横の長さが何種類かある

- 縦の長さは均一
- 横の長さが何種類かある

LIB\_BUF\_X1

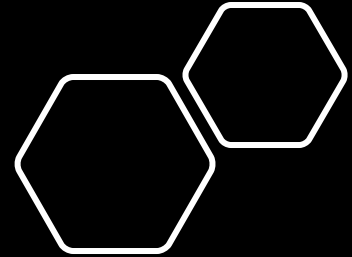
```

L1B, INV_L1B, NAND2B, NAND2B, NAND2B, NAND2B, INV_L1B, AND2_L1B, AND2_X1IB, OR2IB, INV_L1B, AND2_L1B, INV_L1B, INV_L1B, AND2_X1IB, OR2

```

Library: IP62\_5\_stdcell

| No | Cell       | 種類        | 入力ピン数 | 出力ピン数 | セル高さ[um] | セル幅[um] | MP W[um]  | MP L[um] | MN W[um] | MN L[um] |
|----|------------|-----------|-------|-------|----------|---------|-----------|----------|----------|----------|
| 1  | AND2_X1    | ANDゲート    | 2     | 1     | 55.0     | 22.0    | 10.2      | 1        | 3.4      | 1        |
| 2  | AND3_X1    | ANDゲート    | 3     | 1     | 55.0     | 27.5    | 10.2      | 1        | 3.4      | 1        |
| 3  | AND4_X1    | ANDゲート    | 4     | 1     | 55.0     | 33.0    | 10.2      | 1        | 3.4      | 1        |
| 4  | BUF_X1     | バッファ      | 1     | 1     | 55.0     | 16.5    | 10.2      | 1        | 3.4      | 1        |
| 5  | BUF_X2     | バッファ      | 1     | 1     | 55.0     | 22.0    | 10.2 x 2  | 1        | 3.4 x 2  | 1        |
| 6  | BUF_X4     | バッファ      | 1     | 1     | 55.0     | 33.0    | 10.2 x 4  | 1        | 3.4 x 4  | 1        |
| 7  | BUF_X8     | バッファ      | 1     | 1     | 55.0     | 49.5    | 10.2 x 8  | 1        | 3.4 x 8  | 1        |
| 8  | BUF_X12    | バッファ      | 1     | 1     | 55.0     | 66.0    | 10.2 x 12 | 1        | 3.4 x 12 | 1        |
| 9  | BUF_X16    | バッファ      | 1     | 1     | 55.0     | 93.5    | 10.2 x 16 | 1        | 3.4 x 16 | 1        |
| 10 | CLKBUF_X1  | クロックバッファ  | 1     | 1     | 55.0     | 38.5    | 10.2 x 5  | 1        | 3.4 x 5  | 1        |
| 11 | CLKBUF_X2  | クロックバッファ  | 1     | 1     | 55.0     | 66.0    | 10.2 x 10 | 1        | 3.4 x 10 | 1        |
| 12 | CLKBUF_X4  | クロックバッファ  | 1     | 1     | 55.0     | 115.5   | 10.2 x 20 | 1        | 3.4 x 20 | 1        |
| 13 | CLKBUF_X8  | クロックバッファ  | 1     | 1     | 55.0     | 203.5   | 10.2 x 40 | 1        | 3.4 x 40 | 1        |
| 14 | CLKBUF_X12 | クロックバッファ  | 1     | 1     | 55.0     | 291.5   | 10.2 x 60 | 1        | 3.4 x 60 | 1        |
| 15 | CLKBUF_X16 | クロックバッファ  | 1     | 1     | 55.0     | 379.5   | 10.2 x 80 | 1        | 3.4 x 80 | 1        |
| 16 | DEL1       | デレイセル     | 1     | 1     | 55.0     | 38.5    | 10.2 x 2  | 1        | 3.4 x 2  | 1        |
| 17 | DEL2       | デレイセル     | 1     | 1     | 55.0     | 60.5    | 10.2      | 1        | 3.4      | 1        |
| 18 | DEL4       | デレイセル     | 1     | 1     | 55.0     | 104.5   | 10.2      | 1        | 3.4      | 1        |
| 19 | DFFR       | Dフリップフロップ | 3     | 2     | 55.0     | 88.0    | 10.2      | 1        | 3.4      | 1        |
| 20 | DFFS       | Dフリップフロップ | 3     | 2     | 55.0     | 88.0    | 10.2      | 1        | 3.4      | 1        |
| 21 | INV_X1     | インバータ     | 1     | 1     | 55.0     | 16.5    | 10.2      | 1        | 3.4      | 1        |
| 22 | INV_X2     | インバータ     | 1     | 1     | 55.0     | 16.5    | 10.2 x 2  | 1        | 3.4 x 2  | 1        |
| 23 | INV_X4     | インバータ     | 1     | 1     | 55.0     | 27.5    | 10.2 x 4  | 1        | 3.4 x 4  | 1        |
| 24 | INV_X8     | インバータ     | 1     | 1     | 55.0     | 44.0    | 10.2 x 8  | 1        | 3.4 x 8  | 1        |
| 25 | INV_X12    | インバータ     | 1     | 1     | 55.0     | 60.5    | 10.2 x 12 | 1        | 3.4 x 12 | 1        |
| 26 | INV_X16    | インバータ     | 1     | 1     | 55.0     | 77.0    | 10.2 x 16 | 1        | 3.4 x 16 | 1        |
| 27 | MUX2       | マルチプレクサ   | 3     | 1     | 55.0     | 49.5    | 10.2      | 1        | 3.4      | 1        |
| 28 | NAND2      | NANDゲート   | 2     | 1     | 55.0     | 16.5    | 10.2      | 1        | 3.4      | 1        |
| 29 | NAND3      | NANDゲート   | 3     | 1     | 55.0     | 22.0    | 10.2      | 1        | 3.4      | 1        |
| 30 | NAND4      | NANDゲート   | 4     | 1     | 55.0     | 27.5    | 10.2      | 1        | 3.4      | 1        |
| 31 | NOR2       | NORゲート    | 2     | 1     | 55.0     | 16.5    | 10.2      | 1        | 3.4      | 1        |
| 32 | NOR3       | NORゲート    | 3     | 1     | 55.0     | 22.0    | 10.2      | 1        | 3.4      | 1        |
| 33 | NOR4       | NORゲート    | 4     | 1     | 55.0     | 27.5    | 10.2      | 1        | 3.4      | 1        |
| 34 | OR2        | ORゲート     | 2     | 1     | 55.0     | 22.0    | 10.2      | 1        | 3.4      | 1        |
| 35 | OR3        | ORゲート     | 3     | 1     | 55.0     | 27.5    | 10.2      | 1        | 3.4      | 1        |
| 36 | OR4        | ORゲート     | 4     | 1     | 55.0     | 33.0    | 10.2      | 1        | 3.4      | 1        |
| 37 | XNOR2      | XNORゲート   | 2     | 1     | 55.0     | 33.0    | 10.2      | 1        | 3.4      | 1        |
| 38 | XOR2       | XORゲート    | 2     | 1     | 55.0     | 33.0    | 10.2      | 1        | 3.4      | 1        |



VDDライン

P-FET  
Sが共通

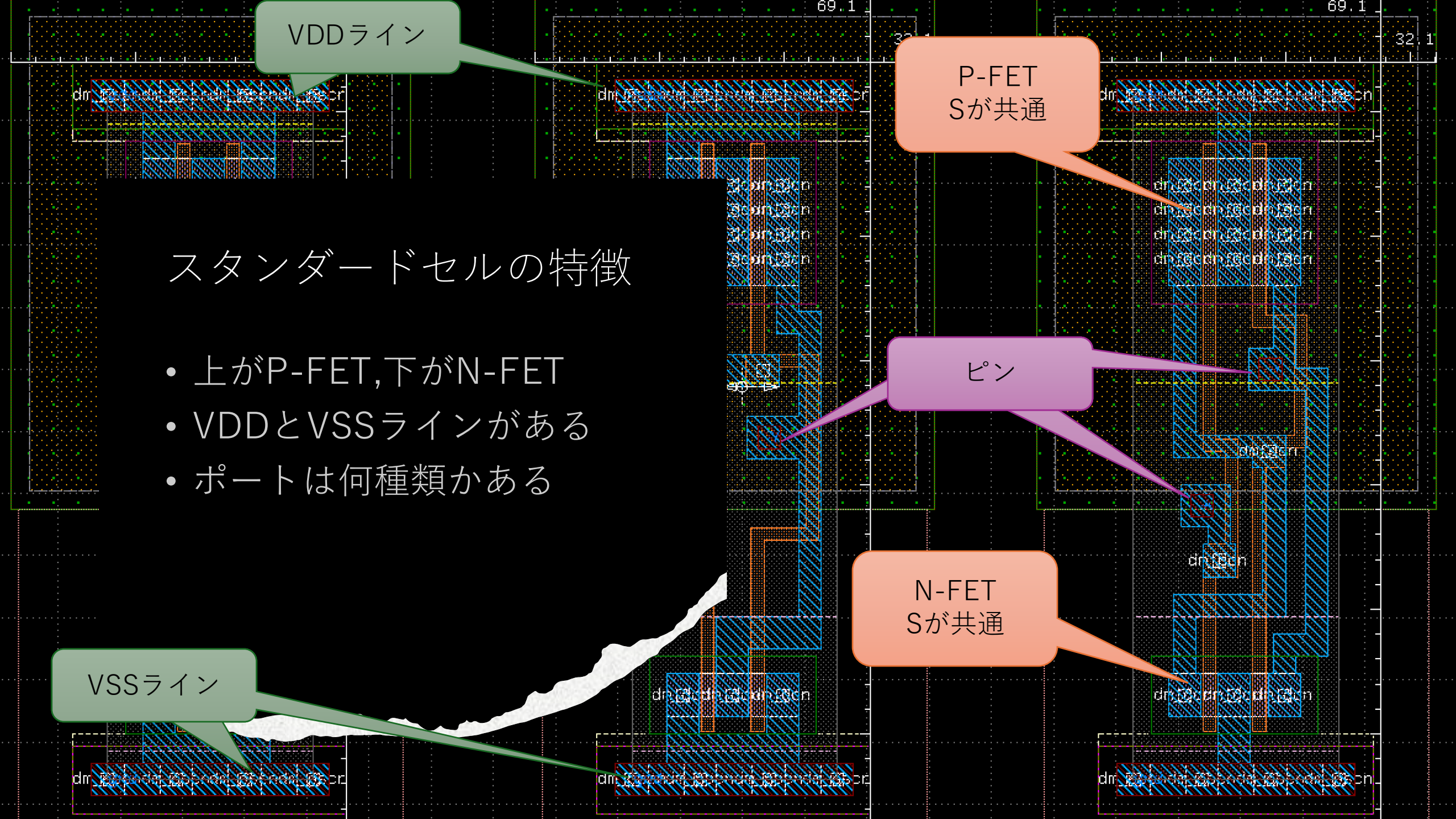
## スタンダードセルの特徴

- 上がP-FET,下がN-FET
- VDDとVSSラインがある
- ポートは何種類もある

ピン

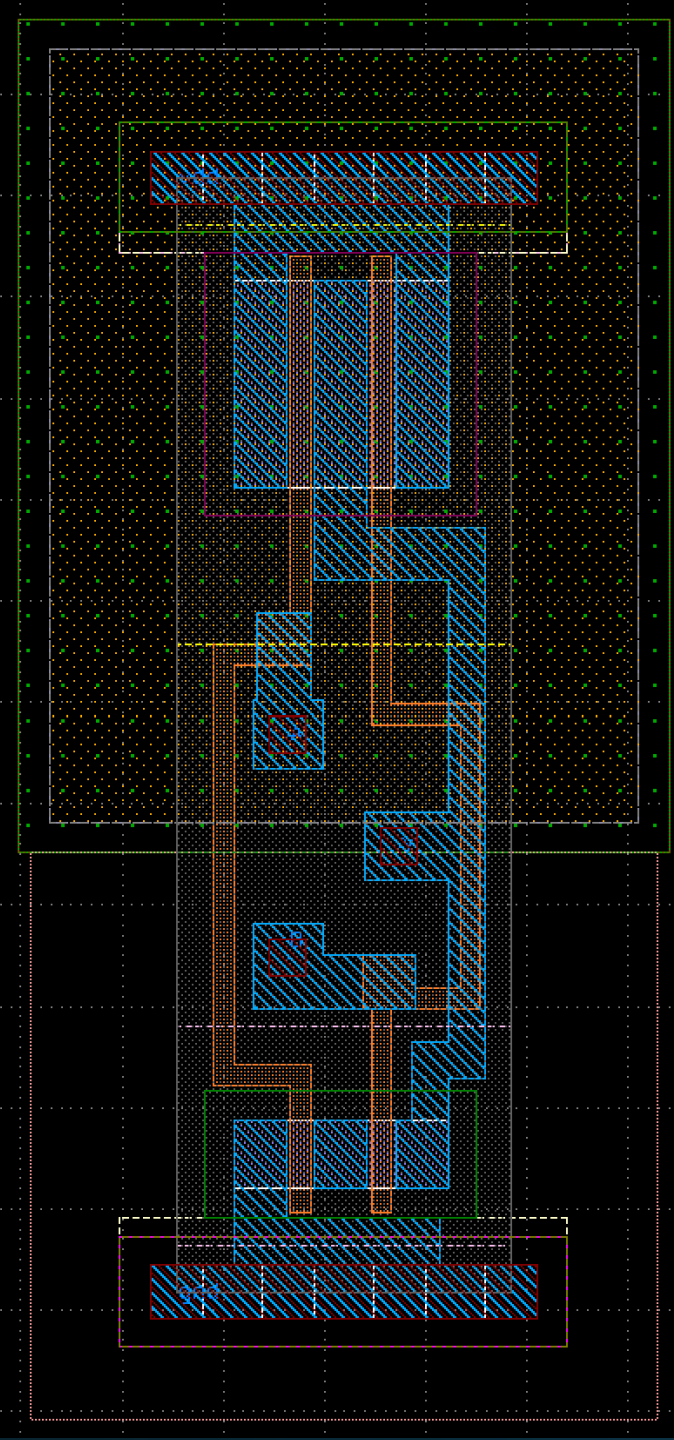
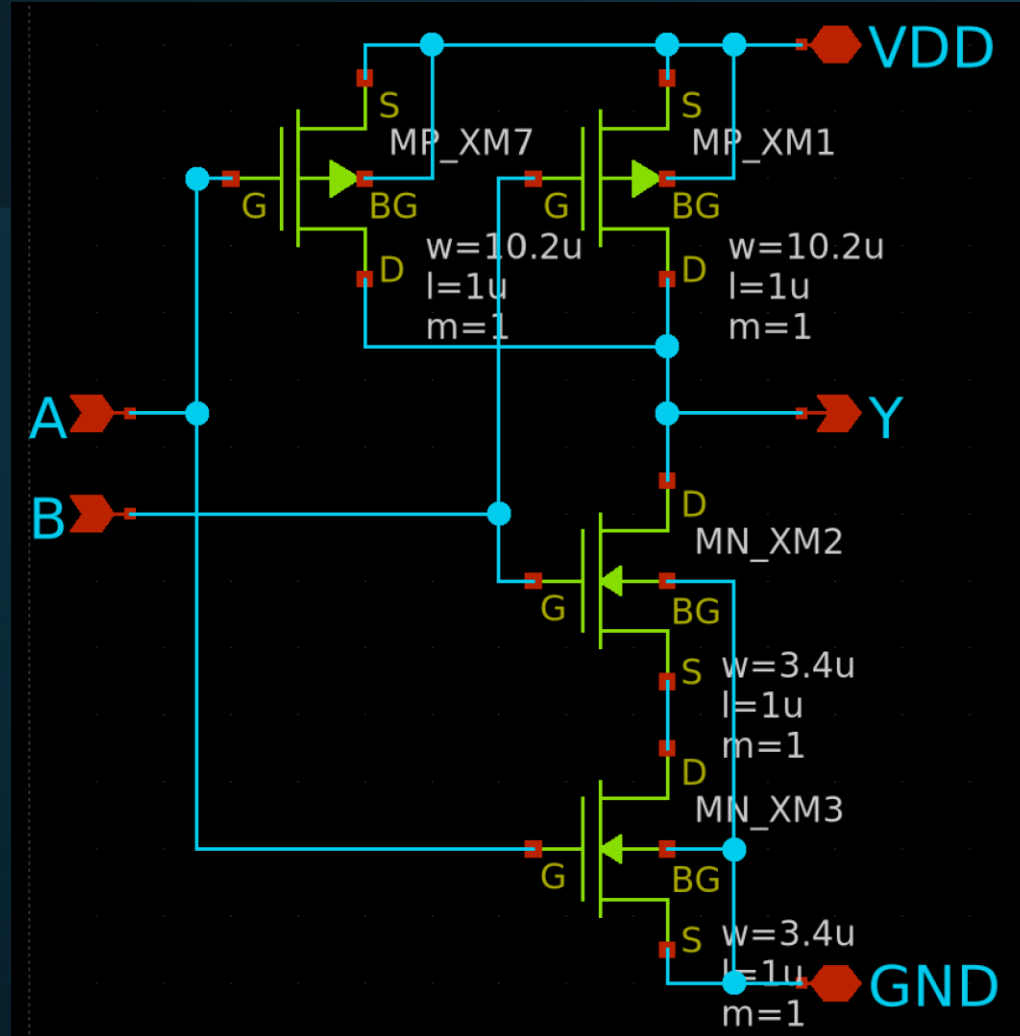
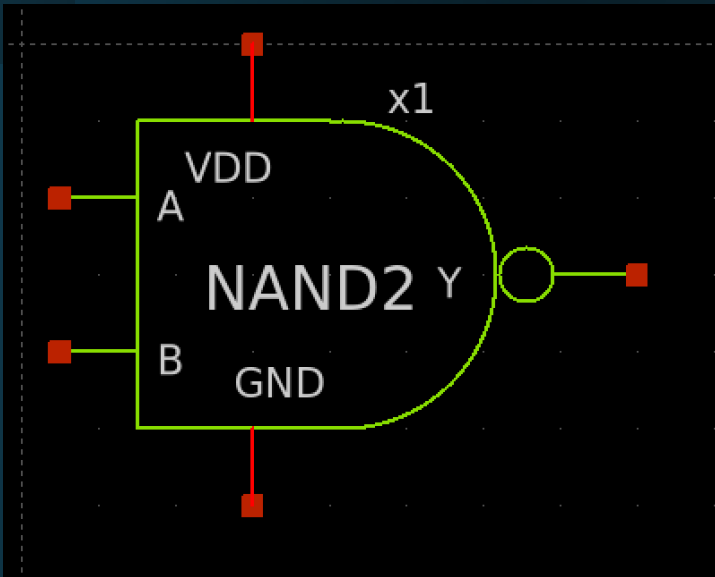
N-FET  
Sが共通

VSSライン





# スタンダードセル



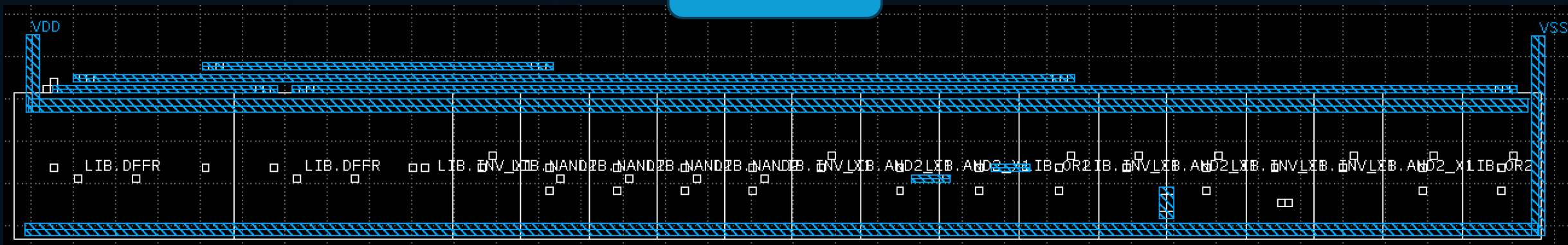
# グリッド配線 (電源グリッド)

- メタル層を縦と横で分ける
- 6層などある場合は6層、5層をVDD, VSSに割り当てる→電源グリッドと呼ばれる
  - 理由：上部の層の方が線厚が厚いため

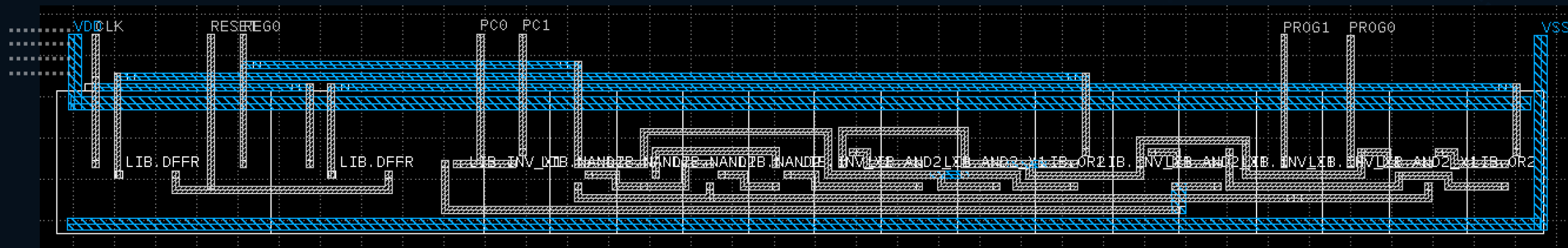
2層グリッド



1層グリッド



# 完成図





# タイミング解析

広島大学・西澤先生の資料より



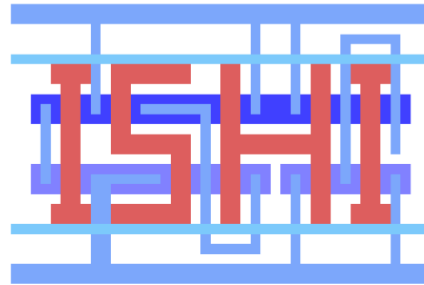
# libretto: An open-source library characterizer for open-source VLSI design

---

Shinichi Nishizawa, Hiroshima Univ., [nishizawa@hiroshima-u.ac.jp](mailto:nishizawa@hiroshima-u.ac.jp)



広島大学



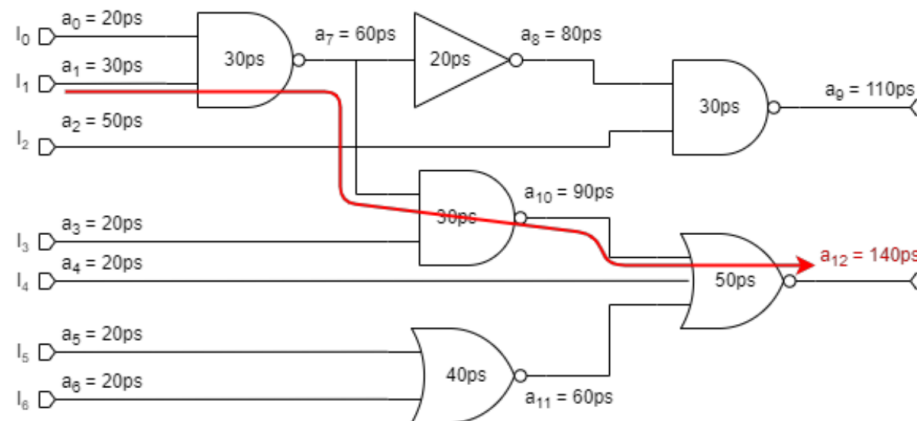
ISHI-Kai (Japan)



Open Source EDA  
supporters (Japan)

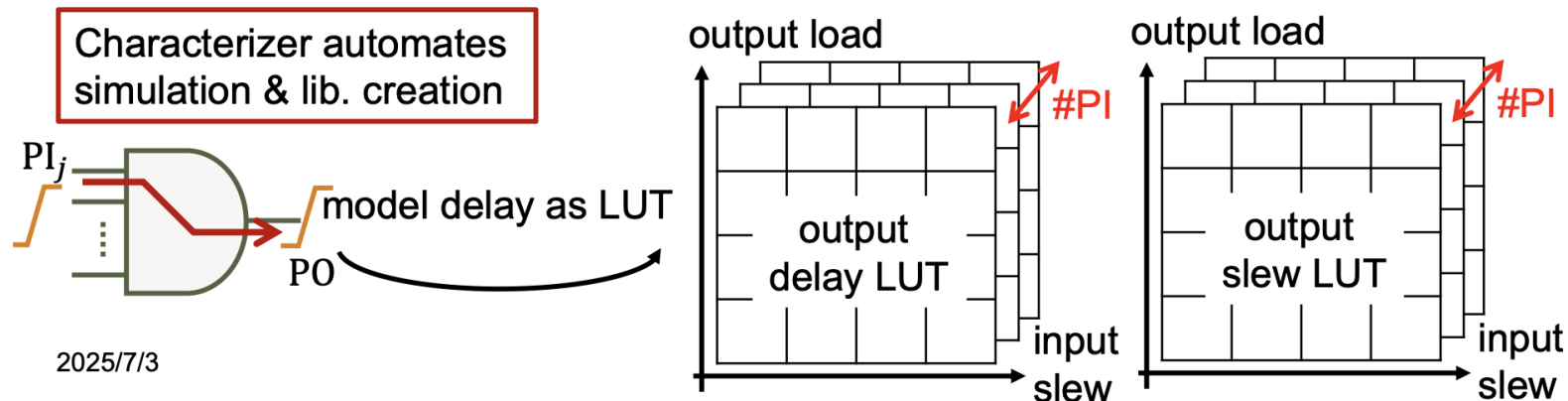
# Digital circuit design and STA

- Static Timing Analysis (STA): estimation of path delay
  - Calculate max/min path delay integrating the cells delay
    - Max delay of  $i$ -th node:  $a_i = \max_{j \in \text{fanin}(i)} \{a_j\} + t_{pd,i}$
  - Need both timing information (.lib) calculation engine (STA)
    - STA: OpenSTA (Open-ROAD), sta (yosis)
    - **.lib** (Liberty): By characterizer



# Characterizer

- Simulate and extract delay and power of std. cell
  - ▣ Prop delay:  $t_{pd} = \text{xx ps}$ . Trans delay:  $t_{td} = \text{xx ps}$
  - ✗ Arrival time:  $a_i = \max_{j \in \text{fanin}(i)} \{a_j\} + t_{pd,i}$
- Delay and energy are the function of input slope, output load
  - ✗ Arrival time:  $a_i = \max_{j \in \text{fanin}(i)} \{a_j\} + t_{pd,i}(t_{pd,PI_j}, C_{load,i})$
- Each primary input (PI) has different delay and energy
  - Arrival time:  $a_i = \max_{j \in \text{fanin}(i)} \{a_j\} + \underline{t_{pd,i,PI_j}(t_{pd,PI_j}, C_{load,i})}$



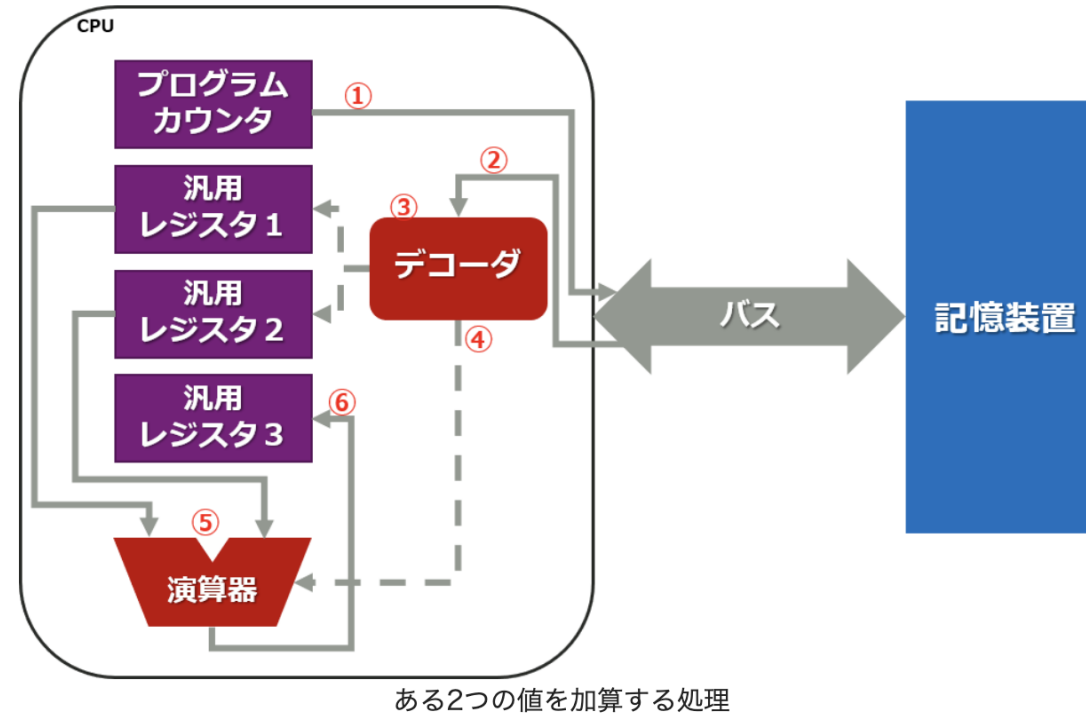
1bit-CPU編

The background features three overlapping teal-colored circles on a dark gray background. A horizontal white band is positioned across the middle of the image, containing the title text.

# CPUの構成

## デコーダ、演算器、内部レジスタのまとめ

これまで説明したデコーダ、演算器、内部レジスタの3つのポイントを元に、「ある2つの値を加算する処理」の流れを確認してみます。



1. 記憶装置から取り出す命令は、プログラムカウンタの値を参考にします。
2. 記憶装置に保存された命令を、実際に取り出します。(Fetch)
3. 取り出した命令を解読し、命令の内容を特定します。(Decode)
4. 解読した命令に従って、汎用レジスタと演算器に、加算処理を行うように制御します。(Execute)
5. 演算器は、汎用レジスタ1と2からデータを取り出して、2つのデータの加算を行います。
6. 計算結果を、汎用レジスタ3に格納します。

以上の1~6の順序で処理が行われることで、「ある2つの値を加算する処理」を実現することができます。

出典：<https://emb.macnica.co.jp/articles/13347/>

CPUとは？

# 1bit-CPU

| 項目        | スペック          |
|-----------|---------------|
| 汎用レジスタ    | 1bit x 1      |
| アドレス空間    | 2bit          |
| アドレスバス幅   | 1bit          |
| ROM容量     | 4bit          |
| 命令セット     | ADD, JMP      |
| プログラムカウンタ | 1bit          |
| フラグレジスタ   | 未実装           |
| 算術演算      | 1bitの加算 (XOR) |

<https://github.com/naoto64/1bit-CPU>

PC

Impress Watch

INTERNET

PC

デジカメ

AKIBA

AV

家電

ゲータイ

クラウド

Watch

窓の杜

こどもとIT

Car

トラベル

グルメ

GAME

HOBBY

MANGA

パソコン工房

### 「世界トップクラスの低性能」 1bit CPU、新発売

劉 堯 2023年12月15日 09:49

✕

ポスト

リスト

f

シェア

B!

はてブ

note

in

LinkedIn

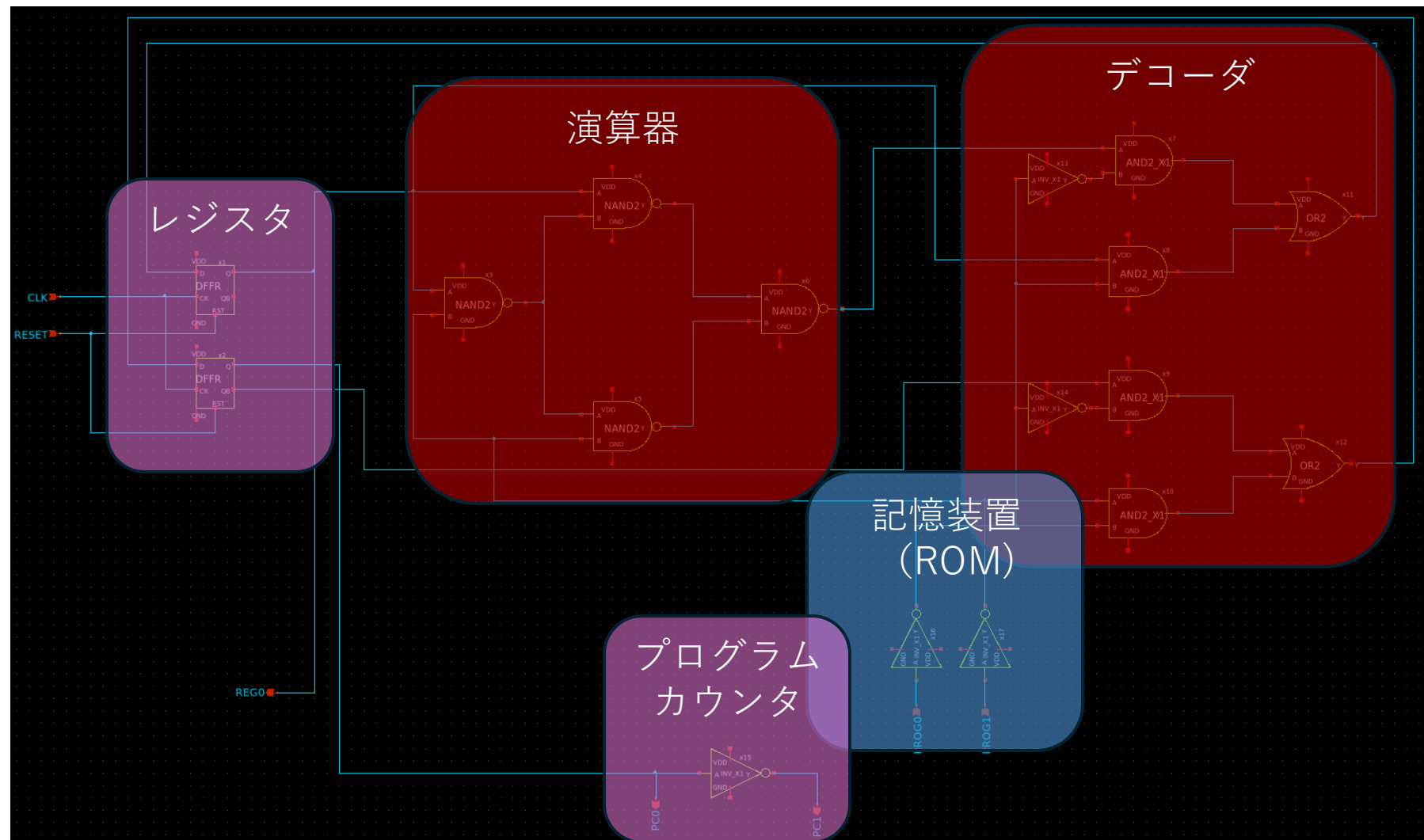


1bit CPU 組み立てキット(実際は自身がはんだ付けなどを行なう必要がある)

### 命令セット

| 命令        | 機械語 | 説明                           |
|-----------|-----|------------------------------|
| ADD A, Im | 0   | AレジスタにIm (イミディエイトデータ) を加算する。 |
| JMP Im    | 1   | Imで指定した先の番地へジャンプする。          |

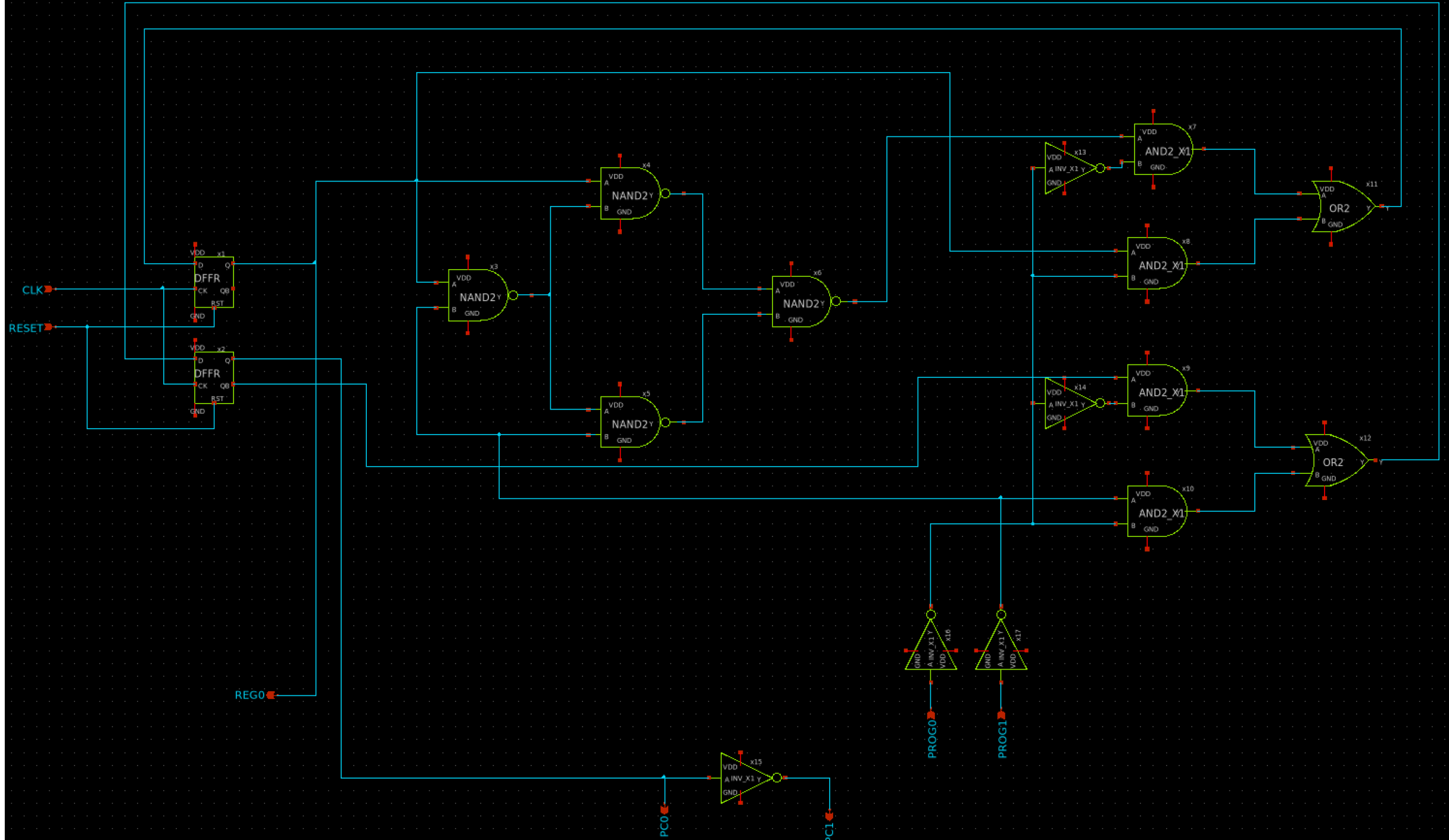
# 回路図







# 回路図







レイアウト

# レイアウト完成例

